Lab 10 - Image Segmentation and Feature Extraction

Course: ARTI404 - Image Processing

College of Computer Science and Information Technology

Imam Abdulrahman Bin Faisal University

This notebook contains:

1. The two examples from the lab manual (Canny Edge Detection, Harris Corner Detection).
2. The assessment task (Otsu's thresholding).

Setup - Import libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data   # provides sample images (camera, checkerboard, coins, ...)

# Make plots show inside the notebook
%matplotlib inline
```

Task 1 - Canny Edge Detection

An edge is where brightness changes sharply. Canny (1986) finds edges in 5 steps: Gaussian blur → gradients → non-maximum suppression (thin the edges) → double thresholding → hysteresis (keep weak edges only if they connect to strong ones).

cv2.Canny(img, T_low, T_high):

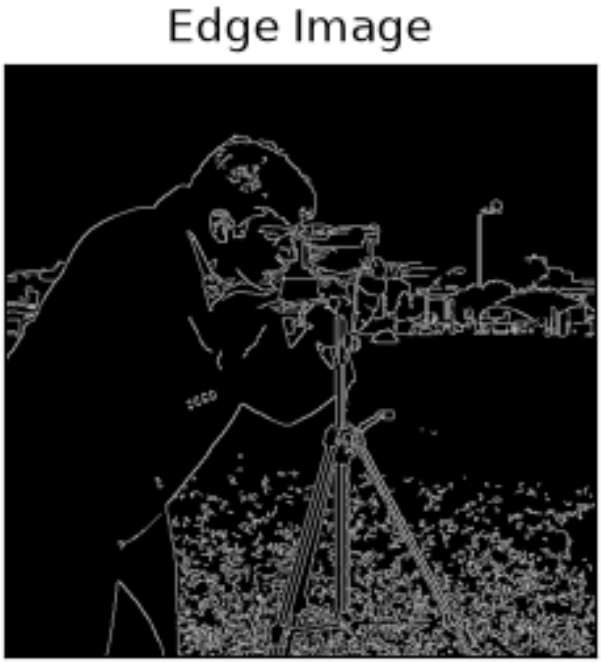
- T_low = lower threshold (weak edges)
- T_high = upper threshold (strong edges)
- Rule of thumb: T_high ≈ 2×T_low or 3×T_low.

```
# --- Code from the lab manual ---
img = data.camera()

edges = cv2.Canny(img, 100, 200)

plt.subplot(121), plt.imshow(img, cmap='gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(edges, cmap='gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()
```





Task 2 - Harris Corner Detection

A corner is a point where the image changes in two different directions - which makes corners highly distinctive (useful for matching, tracking, camera calibration).

- Large positive R → corner
- Large negative R → edge
- Small R → flat region

Parameters: blockSize (window size), ksize (Sobel kernel size), k (sensitivity, typically 0.04-0.06).

- Note: OpenCV uses BGR order, so (0, 0, 255) is red.

```
# --- Code from the lab manual ---
board = data.checkerboard()
image = cv2.cvtColor(board, cv2.COLOR_GRAY2BGR)
image = cv2.resize(image, (400, 400), interpolation=cv2.INTER_NEAREST)

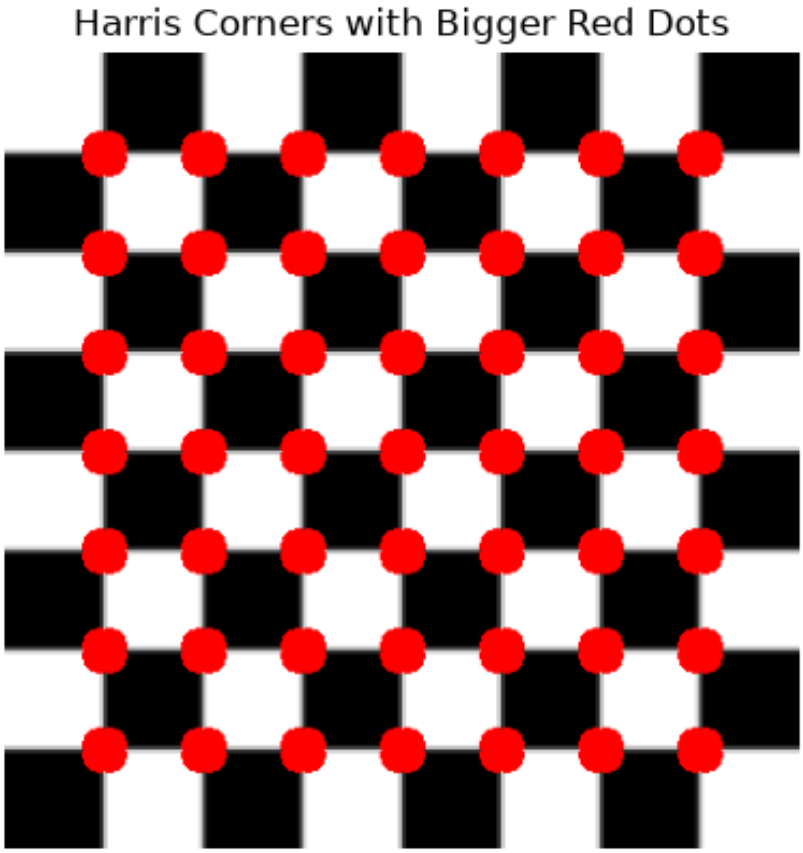
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

# Harris Corner Detection
harris = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None)   # to mark corners more clearly

# Threshold for detecting strong corners
threshold = 0.01 * harris.max()
corner_coords = np.argwhere(harris > threshold)

# Draw larger red dots (radius = 7)
for y, x in corner_coords:
    cv2.circle(image, (x, y), radius=7, color=(0, 0, 255), thickness=-1)  # -1 = filled

# Display
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Harris Corners with Bigger Red Dots')
plt.axis('off')
plt.show()
```



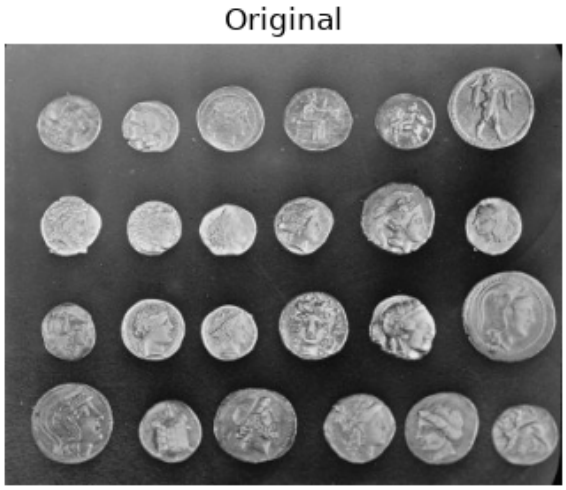
Simple (Global) Binary Thresholding

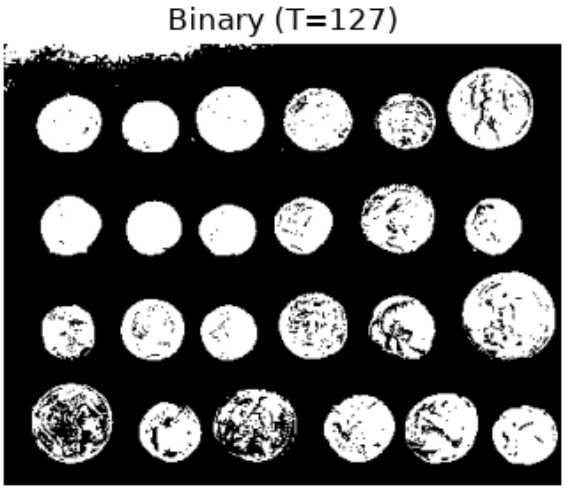
The most basic segmentation: pick a fixed threshold T and turn every pixel above it white, below it black.

```
img = data.coins()

T = 127
_, binary = cv2.threshold(img, T, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(9, 3))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.axis('off')
plt.subplot(1, 2, 2), plt.imshow(binary, cmap='gray'), plt.title(f'Binary (T={T})'),
plt.axis('off')
plt.tight_layout(), plt.show()
```





Q1 - Otsu's Thresholding (Assessment Task)

Otsu picks T automatically by maximizing the between-class variance - assumes the histogram has two peaks (foreground + background).

Show the original image, its histogram, and the thresholded image.

```
# Apply Otsu's method to choose the threshold automatically
otsu_threshold, binary_otsu = cv2.threshold(
    img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Display the original image, its histogram, and the Otsu result
fig, ax = plt.subplots(1, 3, figsize=(14, 4))

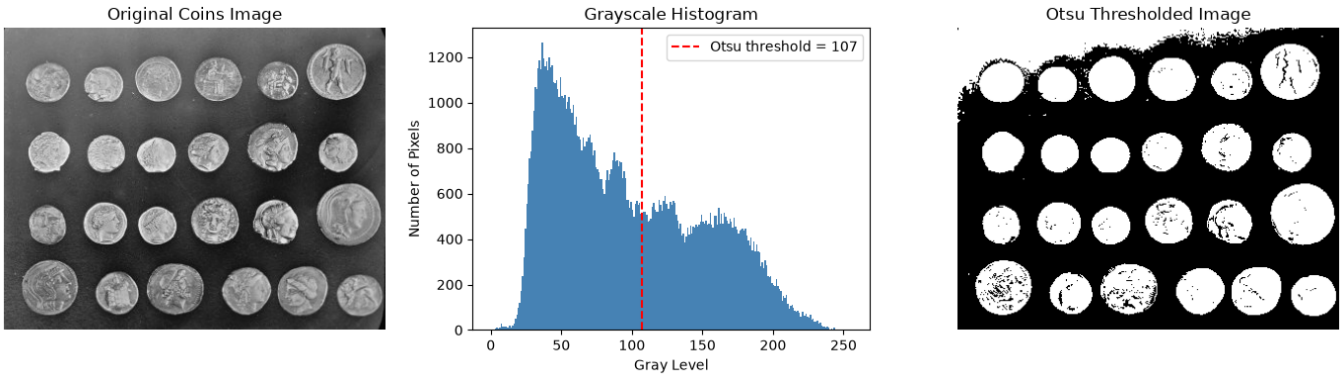
ax[0].imshow(img, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Coins Image")
ax[0].axis("off")

ax[1].hist(img.ravel(), bins=256, range=(0, 256), color="steelblue")
ax[1].axvline(otsu_threshold, color="red", linestyle="--",
              label=f"Otsu threshold = {otsu_threshold:.0f}")
ax[1].set_title("Grayscale Histogram")
ax[1].set_xlabel("Gray Level")
ax[1].set_ylabel("Number of Pixels")
ax[1].legend()

ax[2].imshow(binary_otsu, cmap="gray", vmin=0, vmax=255)
ax[2].set_title("Otsu Thresholded Image")
ax[2].axis("off")

plt.tight_layout()
plt.show()

foreground_pixels = np.count_nonzero(binary_otsu)
foreground_percent = 100 * foreground_pixels / binary_otsu.size
print(f"Otsu threshold: {otsu_threshold:.0f}")
print(f"White foreground pixels: {foreground_pixels} ({foreground_percent:.1f}%)")
```




```
Otsu threshold: 107
White foreground pixels: 45117 (38.8%)
```

Summary table

Technique	Category	What it finds	OpenCV function
Canny	Edge detection	Thin connected edges	cv2.Canny
Sobel	Edge detection	Gradient magnitude (thick)	cv2.Sobel
Laplacian	Edge detection	Zero-crossings (2nd deriv)	cv2.Laplacian
Harris	Feature extraction	Corners	cv2.cornerHarris
Shi-Tomasi	Feature extraction	Corners (improved)	cv2.goodFeaturesToTrack
Simple threshold	Segmentation	Fixed-T binary mask	cv2.threshold
Otsu	Segmentation	Auto bimodal threshold	cv2.threshold + THRESH_OTSU
Adaptive threshold	Segmentation	Local-T binary mask	cv2.adaptiveThreshold

End of notebook.